APMA 4302 Methods - Homework 3: Non-linear solver for reaction-diffusion equations

Marc Spiegelman

March 23, 2026

Here we will solve the nonlinear reaction-diffusion equation

$$-\nabla^2 u + \gamma u^p = f$$

on the unit square with non-homogeneous Dirichlet boundary conditions. Here γ is a positive constant and $p > 1$ is an integer. The right-hand side f will be determined by the method of manufactured solutions, assuming the exact solution is given by

$$u_{exact}(x, y) = A \exp\left(-\frac{(x - x_0)^2 + (y - y_0)^2}{\sigma^2}\right),$$

where (x_0, y_0) is a point in the unit square and σ is a positive constant controlling the width of the Gaussian of maximum amplitude A.

To help with this exercise, I have provided the code `poisson2d.c` in the `apma4302-methods` repository which solves the linear version of this problem (i.e. $\gamma = 0$) as the discrete problem $A\mathbf{x} = \mathbf{b}$ as well as providing `VTK` output for visualizing in ParaView.

Do the following:

1. (**5 pts**) Use finite differences to discretize the nonlinear PDE and write down the resulting nonlinear system of equations in the form $F(\mathbf{u}) = 0$. Include auxiliary equations for non-homogeneous Dirichlet boundary conditions.

$$F(u) = -\nabla^2 u - \gamma u^p - f = 0 \tag{1}$$

To discretize this equation, define N_x, N_y as the number of points in the x-direction and y-direction of the discrete grid, respectively. The grid spacing is then:

$$h_x = \frac{1}{N_x - 1} \tag{2}$$

$$h_y = \frac{1}{N_y - 1} \tag{3}$$

Using second-order finite differences which results in a 5-point stencil for the 2D Laplacian, the discretization for $F(u)$ looks like the following.

$$\frac{h_x}{h_y}\begin{bmatrix} \cdots & -u_{i-1,j} & 0 \\ -u_{i,j-1} & 4u_{i,j} & -u_{i,j+1} \\ 0 & -u_{i+1,j} & \cdots \end{bmatrix} + h_x\gamma u_{i,j}^p - h_x f_{i,j} = 0 \tag{4}$$

In this equation immediately above, i,j are the row,column indices, respectively. The matrix exists along the diagonal of the whole matrix, and all other non-specified values by this matrix cutout are zero. The γ and f terms are known matrices determined by the values of u and f at each i, j combination. The boundary conditions are not included yet, and Eq. 4 is assumed to apply to all elements except those on the edges of the matrix (where $i = 0, N_y - 1$ or $j = 0, N_x - 1$). We can explicitly define these boundary conditions with the following equations:

$$u_{0,j} = b_j \tag{5}$$

$$u_{i,0} = c_i \tag{6}$$

$$u_{N_y-1,j} = r_j \tag{7}$$

$$u_{i,N_x-1} = s_i \tag{8}$$

where b_j, c_i, r_j, s_i are functions with values along i, j, respectively.

2. (**5 pts**) Provide equations for elements of the Jacobian matrix $J_{i,j}(\mathbf{u}) = \frac{\partial F_i}{\partial u_j}$.

For a problem on a 1D grid, the Jacobian is 2D. For a problem on a 2D grid, the Jacobian is 4D and is defined as follows.

$$J_{(i,j),(k,l)} = \frac{\partial F_{(i,j)}}{\partial u_{(k,l)}} \tag{9}$$

The Jacobian has diagonal elements when $(i, j) = (k, l)$. This manifests as the derivative with respect to u of the diagonal elements of the discretized $F(u)$.

$$J_{(i,j),(i,j)} = \frac{\partial F_{(i,j)}}{\partial u_{(i,j)}} = 4\frac{h_x}{h_y} + \gamma h_x p u^{p-1} \tag{10}$$

The Jacobian has off-diagonal elements when $(i, j) = (i \pm 1, j)$ and $(i, j) = (i, j \pm 1)$ due to the shape of the 5-point stencil in Eq. 4. This takes the form:

$$J_{(i,j),(i\pm1,j)} = \frac{\partial F_{(i,j)}}{\partial u_{(i\pm1,j)}} = -\frac{h_x}{h_y} \tag{11}$$

$$J_{(i,j),(i,j\pm1)} = \frac{\partial F_{(i,j)}}{\partial u_{(i,j\pm1)}} = -\frac{h_x}{h_y} \tag{12}$$

At the boundaries, $F_{(i,j)} = u_{(i,j)} - g_{(i,j)}$ where g is a general function describing the relevant boundary conditions for whatever edge you are at (compactifying the notation used in Part 1 of this problem). Thus, the boundaries have Jacobian:

$$J^{boundary}_{(i,j),(k,l)} = \frac{\partial(u_{(i,j)} - g_{(i,j)})}{\partial u_{(k,l)}} = \begin{cases} 1 & , (i,j) = (k,l) \\ 0 & , else \end{cases} \tag{13}$$

Since all other elements of $F(u)$ are zero, all other terms in the Jacobian are zero.

3. (**20 pts**) Modify the code `reaction.c` in the `p4pdes` repository to solve the nonlinear problem using Newton's method with the `PETSc SNES` objects. Your code should have the following properties.

   (a) It should run with the interface

   ```
   mpirun -n <num_procs> ./reaction2d  -rct_p <p>
           -rct_gamma <gamma> -rct_linear_f
   ```

   plus standard petsc command-line options.

   The option `-rct_linear_f` should be a boolean flag such that if true, sets the right-hand side to be

   $$f(x,y) = -\nabla^2 u_{exact}$$

   (i.e. the RHS for the linear problem) and if false, sets the right-hand side to be the full nonlinear RHS for the manufactured solution

   $$f(x,y) = -\nabla^2 u_{exact} + \gamma u^p_{exact}$$

   You can use the code in `poisson2d.c` to calculate u_{exact} and its derivatives.

   (b) Boundary conditions should be u_{exact} on all boundaries, independent of the value of `-rct_linear_f`.

   (c) It should include routines to compute both the nonlinear residual $F(\mathbf{u})$ and the Jacobian matrix $J(\mathbf{u})$.

   (d) It should also be able to run with Finite difference jacobians using the options `-snes_fd`, `-snes_fd_color`, `-snes_mf` and `-snes_mf_operator` and should give the same results as the user-provided Jacobian. (you can use the option `-snes_test_jacobians` to check.

   (e) It should output the solution in `VTK` format for visualization in `ParaView`.

   NOTE: There is an important *but* here that one of the AI's caught when I was unable to achieve a good relative error on u. The use of what I have prescribed as the Jacobian and residual is using a 5-point stencil that is second-order in grid spacing, while the evaluation of the boundary conditions (since they are using the "exact" conditions) is order one in grid spacing. This creates a mis-match at the boundaries and causes the relative error to spike. With the help of this AI I switch the order-two scheme to an order-one scheme, which is now implemented in the code I am submitting in this homework.

4. (**5 pts**) Run your code with $\gamma = 0$ for a 65×65 grid and show that the linear and nonlinear solvers produce the same solution within error. Find an appropriate set of linear solver options and stopping

conditions so that the non-linear solver converges to a residual norm $\|F(\mathbf{u})\|_2 < 10^{-10}$ in one newton iteration. Report the number of linear iterations taken at each Newton step and the final *relative error* $\|\mathbf{u} - \mathbf{u}_{exact}\|_2/\|\mathbf{u}_{exact}\|_2$. (hint: some useful command line options include `-ksp_monitor`, `-snes_monitor`, `-ksp_rtol` and `-ksp_atol`). Feel free to experiment with different linear solvers and preconditioners (including sparse-direct solvers), but your code should run in parallel. Save your preferred options in a file `options_file_gamma0` and include it with your submission.

The residual norm of the non-linear solver can be found by including the flag "-snes_monitor". The number of non-linear Newton iterations will be shown by the outputted number next to the reported SNES function norm.

I have successfully ran my reaction2d code and achieved a SNES residual norm of 1.13e-14 after one non-linear Newton iteration. This required 152 linear iterations (with KSP). The final relative error for this run using a two-norm was 0.000525. The same answer results when I set the rct_linear_f to true.

5. (**2 pts**) Show that you get the same solution with the finite-difference Jacobian and report any differences in convergence behavior or final error. Include the command line options you used to run with the finite-difference Jacobian in a file `options_file_fd` and include it with your submission. Compare the run times for the user-provided Jacobian and the finite-difference Jacobian and report any differences. (hint: use `-log_view` to get detailed timing information).

   We can add the finite difference Jacobian to the run by specifying the flag "-snes_fd".

   I have kept all the other parameters the same from the runs in Part 4 of this homework. When this is run, we now have two SNES iterations that occur. The first iteration has 152 KSP iterations, and the second has 48, finishing with a SNES norm of 1.11e-14 (similar to the previous part of this homework). The final relative error in the solution is found to be 0.000525, which is the same as Part 4 of this homework.

   The run-time for the finite-difference Jacobian was 0.255 seconds. The run-time for the user-provided Jacobian was 0.00362 seconds.

   Note all command line and options behavior are in the tagged Github for this assignment.

6. (**5 pts**) Now set $\gamma = 100$ and $p = 3$ and run the nonlinear solver with the same stopping conditions as before. Report the number of Newton iterations taken, the number of linear iterations taken at each Newton step and the final *relative error* $\|\mathbf{u} - \mathbf{u}_{exact}\|_2/\|\mathbf{u}_{exact}\|_2$. Plot the convergence history of the nonlinear residual norm $\|F(\mathbf{u})\|_2$.

   The number of SNES non-linear iterations taken is now 2. The number of KSP iterations taken at each SNES step, sequentially, is: 96, 60. The final relative error is 0.000203.

   Note my low number of SNES steps is likely because I am taking a very good initial guess, which is the solution u_{exact}. For the convergence plot below in Fig. 1, I am plotting what happens when I instead set my initial guess to zero with the boundary conditions satisfied. This run had 6 SNES iterations (I won't outline each number of KSP steps). The resulting relative error was the same as the other initial condition, 0.000203.

7. (**5 pts**) rerun the problem with `-rct_linear_f` and visualize the solution for $\gamma = 100$ and $p = 3$ in `ParaView` and include a screenshot of the solution in your submission. Describe how the nonlinearity has affected the solution compared to the linear case $\gamma = 0$.

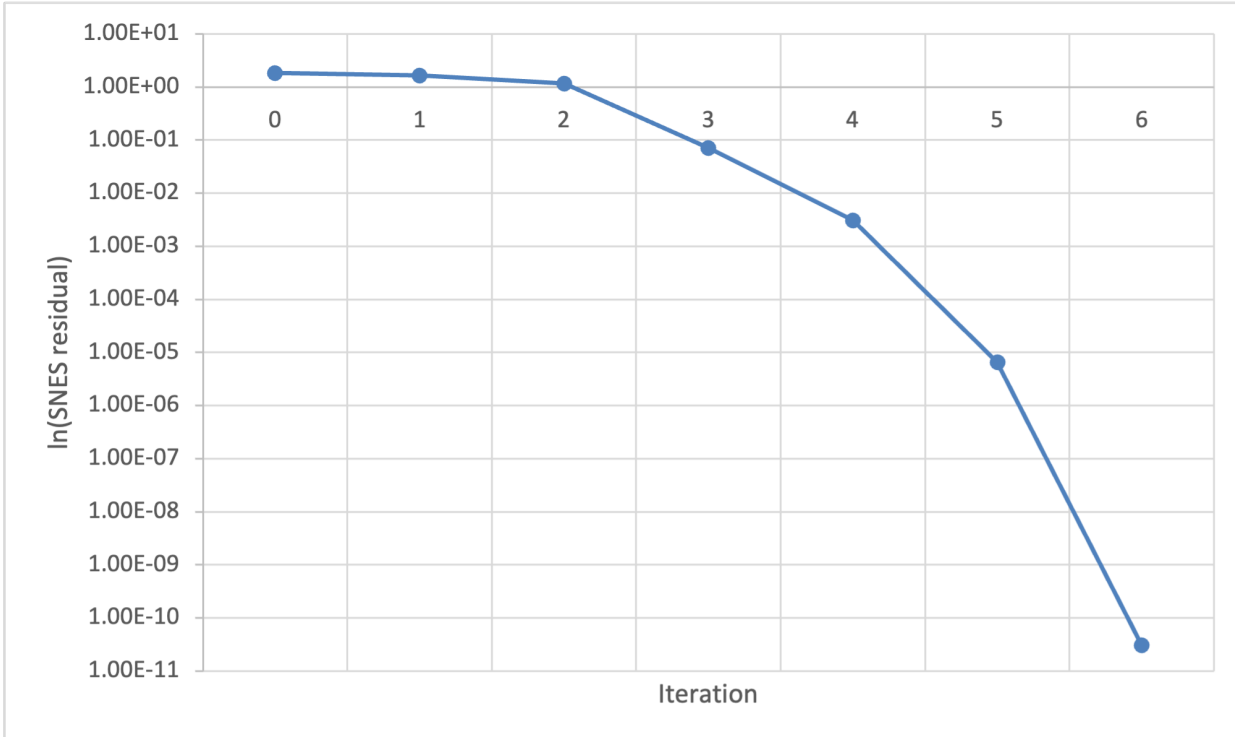
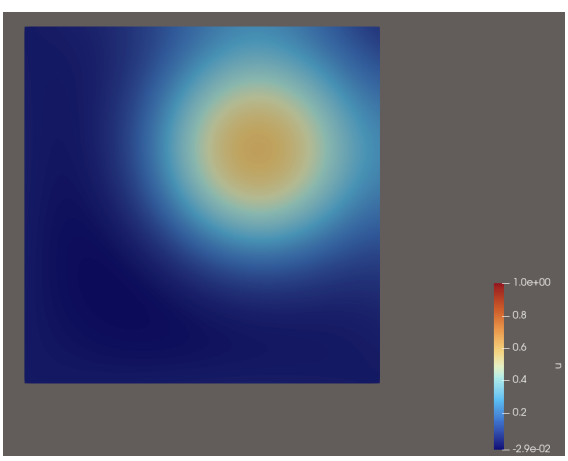
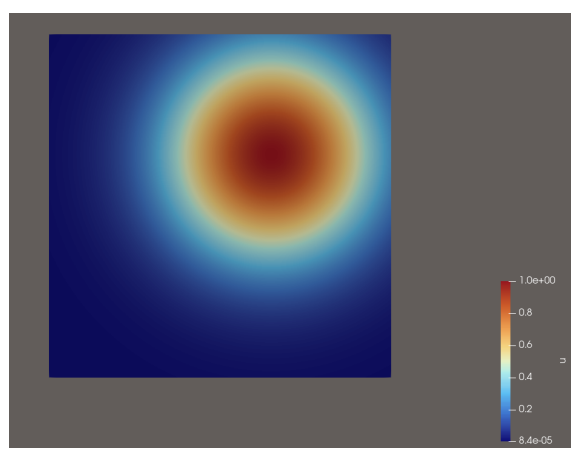


Figure 1: Problem 6 poor initial guess non-linear SNES residual for a non-linear problem.

The pictures below (Fig. 2) detail the linear and non-linear version of the prescribed problem. It seems that in the linear case, there is less of a source, and in the non-linear case this source is heightened. The maximum value of the source is one but the minimum value of the non-linear solution is much smaller than the linear one. I am guessing there is a normalization here that is occuring that I may have applied accidentally.



(a) Linear



(b) Non-linear

Figure 2: Comparison of linear and non-linear flags for the parameters utilized in Problem 6 of this homework.

8. (**10 pts**) **Performance and scaling** make a table (and/or plot) comparing

   - run times (use `-log_view`)
   - number of Newton iterations
   - relative norm of the error $\|\mathbf{u} - \mathbf{u}_{exact}\|_2 / \|\mathbf{u}_{exact}\|_2$

   for the non-linear problem for `-da_refine` $2, 3, 4, 5, 6$ (i.e. grid sizes of 33×33, to 513×513) and 1,2 and 4 processors. I may actually provide a script for this part. Comment on the scaling of the code with respect to both problem size and number of processors.

Grid Size n x n	Number of Processors	Run Time	SNES Iterations	Relative error norm
33	1	0.0301	8	8.10e-4
65	1	0.0522	7	2.03e-4
129	1	0.319	6	5.06e-5
257	1	4.194	8	1.27e-5
513	1	53.710	8	3.17e-6
33	2	0.0106	8	8.10e-4
65	2	0.0388	7	2.03e-4
129	2	0.215	6	5.06e-5
257	2	2.653	8	1.27e-5
513	2	36.920	8	3.17e-6
33	4	0.00700	8	8.10e-4
65	4	0.0280	7	2.03e-4
129	4	0.139	6	1.27e-5
257	4	3.088	8	1.27e-5
513	4	32.430	8	3.17e-6

I don't have the time to make any plots of this although that probably would be a good idea here, so I won't be able to really tell any power-law scalings sorry. The scaling of the number of processors predictably decreases the run time in a possibly square root manner. Number of processes has no effect on the relative error, as expected. The number of SNES iterations is a bit unpredictable and doesn't depend on processor number either, but instead reaches a minimum in the middle of the grid resolution scan. I am unsure what is special about 129x129 grids! Curious. As the grid size increases, the run time jumps up pretty heavily. Crude estimate is giving a n^2 scaling.